

Hybrid XGBoost-LSTM Ensemble with Attention Mechanism for Remaining Useful Life Prediction of Turbofan Engines

Darshan Gowda S

*Dept. of Information Science and Technology,
Presidency University, Bengaluru, India
DARSHAN.20221IST0055@presidencyuniversity.in*

Dr. Afroz Pasha

*School of Computer Science and Engineering,
Presidency University, Bengaluru, India
afrozpasha@presidencyuniversity.in*

Chirag Gowda S V

*Dept. of Information Science and Technology,
Presidency University, Bengaluru, India
CHIRAG.20221IST0112@presidencyuniversity.in*

Chethan C

*Dept. of Information Science and Technology,
Presidency University, Bengaluru, India
CHETHAN.20221IST0069@presidencyuniversity.in*

Abstract—Predictive maintenance of aircraft turbofan engines is a major issue in aerospace engineering whereby effective predictive maintenance of Remaining Useful Life (RUL) may be used to avoid disastrous failure and also to save on operational expenses. In other words, schemes presently exist largely on single-model frameworks of deep learning frameworks that learn temporal relations or classical machine learning frameworks that utilize statistical characteristics and fail to leverage the synergistic advantages of the two paradigms combined. This paper suggests a hybrid ensemble model in the form of an attention-based Long Short-Term Memory (LSTM) network and an Extreme Gradient Boosting (XGBoost) regressor that is used to predict RUL of the turbofan engines. The LSTM attention mechanism takes sensor sequences of 73-dimensional feature vectors (3 operating condition settings, 14 raw sensor readings and 4 statistical features $\times$ 14 sensors = 56 engineered features) through 50 cycles to learn how to degrade over time, whereas XGBoost predicts using the same engineered feature space as viewed statistically. A grid search is used to optimize a dataset-specific blending weight α on a separate validation partition to produce a weighted ensemble prediction. In order to address the problem of multi-condition engine operation in FD002 and FD004, KMeans clustering is used to determine six operating regimes and per-cluster Z-score normalization to de-couple sensor baseline shifts and actual degradation signals. LSTM training uses Huber Loss with $\delta = 10$ so that it is robust to outlier engines. The offered system is tested on the four sub-datasets of NASA C-MAPSS benchmark. Findings have shown that the most successful model depends on the complexity of the sub-datasets: standalone LSTM performs best on FD001 (RMSE = 15.58) and FD003 (RMSE = 13.54), whereas the hybrid ensemble performs better on the multi-condition sub-datasets FD002 (RMSE = 17.47, $R^2 = 0.83$) and FD004 (RMSE = 23.56, $R^2 = 0.70$).

Index Terms—Remaining Useful Life, Predictive Maintenance, LSTM, XGBoost, Hybrid Ensemble, Attention Mechanism, NASA C-MAPSS, Turbofan Engine Degradation, Huber Loss.

I. INTRODUCTION

The safety and reliability standards of the field of engineering are among the most rigid in aviation business. The key to any commercial airliner- the engine that converts fuel to thrust under extreme conditions of thermal and mechanical stress, flight after flight, cycle after cycle, is the turbofan engine. Such stress wears off major internal parts eventually. The High-Pressure

Compressor stages and fan are particularly vulnerable and once these wear is not detected early enough, it leads to the loss of performance, components break down and in worst-case scenarios lives are lost [8].

The economic fact is also immediate. Engine maintenance is the largest percentage of all aviation MRO spending worldwide. Airlines that perform late maintenance can face unlikely grounding and aircraft failures. Airlines that book too early waste resources refurbishing parts that have a lot of life left in them. Both the results are unacceptable in the safety-critical and competitiveness industry. This is the question that was answered by Predictive Maintenance (PdM). Rather than fix a failed engine, or even replace components at a defined cost, PdM uses the sensor data already flowing from the engines to determine how much operational life each individual engine remains of its potential- its Remaining Useful Life (RUL). The maintenance can subsequently be planned at the most optimal time: late enough to allow the components to be utilized fully, yet not too late otherwise the risk of failure is a possibility [2], [6]. Recent developments in deep learning systems that predictive maintenance have further demonstrated the significance of combining model predictions and operational planning models [20].

RUL estimation is mathematically an estimation of the number of remaining number of active cycles before an engine exceeds a set failure threshold, given its sensor history to the current cycle. It is far more challenging than an average regression work. There exist numerous flight regimes (altitude, speed, throttle position) over which engines are operated and sensor baselines vary in a superficially degradation-like manner [10]. Raw sensor values are also distorted by measurement errors and atmospheric variability and the number of full run-to-failure trajectories to be used in training is limited in nature.

Over the last decade, machine learning has achieved actual development in this issue. Initially, random forests and support vector regression have performed well such that competitive RUL predictions could be obtained based on manually designed statistical features [1], [3]. The

breakthrough in LSTM networks was that they were able to learn degradation dynamics using raw time-series sensor measurements [12], [14]. Convolutional, hybrid CNN-LSTM networks went a step further in terms of accuracy [4], [15]. Meanwhile, gradient boosting classifiers such as XGBoost showed comparable performance on tabular representation of features [11].

Nonetheless, there is an eternal gap. Most of the published approaches are dedicated to a single paradigm of modeling. Deep learning models are trained to capture the pattern of wear in terms of their temporal degradation form but are not trained to capture the statistical distribution patterns of wear. Tree-based models utilize the statistical properties and are incapable of reacting to sequential dynamics that signal the beginning of degradation. These two views are truly complementary, but not even a single paper has joined them together with an ensemble strategy based on principles and a blending weight conditioned on a learned dataset. Multi-condition operation is not well-known or an issue well-addressed without a sensible normalization policy [3], [5], and the vast majority of the literature does not relate model outputs to real-life maintenance decision support [6], [7], [9].

The four contributions to these gaps presented in the present paper are the following:

1. Hybrid LSTM-XGBoost ensemble with a per-dataset learned blending weight α , which has been optimized by grid search;
2. An operating condition normalization strategy ($k = 6$) based on the KMeans algorithm to separate the variations in flight regimes and the presence of degradation signals;
3. A soft attention mechanism to prioritize on informative degradation cycles;
4. Huber Loss with $\delta = 10$ to train well on outlier engine trajectories.

The rest of this paper will be structured in the following way. Section II is a literature review. The suggested methodology is outlined in Section III. The fourth section (IV) contains and talks about the results of the experiments. Future directions are also presented in section V.

II. LITERATURE SURVEY

The interest in studying the problem of data-driven RUL estimation has a long-standing research history since its initial release in 2016 by NASA C-MAPSS benchmark, including both classical, deep, and hybrid machine learning methods. The section presents the review of the work most related and the gaps prompting the contribution that is being made.

The antecedent leading to condition-based prognostics was the generic architecture of Jardine et al. [8], which surveyed the signal processing, modeling and decision-making functions of an entire PHM pipeline. The main point that they identify is that efficient prognostics involves not only good models but also good sensor processing and incorporation with maintenance decisions, which continues to be under-researched in much of the

later literature. The benchmark of C-MAPSS proposed by Saxena et al. [10] allows conducting a systematic comparison of prognostic programs in four sub-configurations of different complexity.

In classical paradigm, Patil et al. [1] delineated a predictive maintenance structure based on the utilization of Random Forest, SVM and Decision Trees, which showed competition accuracy with relevant feature choice in industrial contexts. They use observations singly, but they do not take into account the time-course of sensor readings which most directly encode degradation. Sharma et al. [3] conducted a survey of ML-based predictive maintenance in general and found that not a single algorithm can consistently work better than all others in all machine types and operating scenarios a fact that directly inspires the ensemble strategy adopted in this case.

The LSTM networks represented a turning point in the RUL estimation [12]. As it was demonstrated by Zheng et al. [14], LSTMs were able to model turbofan degradation on C-MAPSS with high accuracy, including long-range temporal dependencies, which classical feature-based models were unable to obtain. Li et al. [15] demonstrated that hierarchical temporal feature extraction matched or surpassed LSTM with the help of deep convolutional networks. In more recent work, improved RUL accuracy on the C-MAPSS benchmark has been achieved with hybrid CNN-LSTM network with attention mechanisms [17], [18]. Li and Li [4] compared CNNs, LSTMs, and CNN-LSTMs hybrids in the manufacturing process and discovered that hybrid networks always achieved better results compared to single-type networks- a result that directly forms the basis of the ensemble paradigm of this study.

A system of Aziz and Sowmyashree [7], which uses an Autoencoder to detect anomalies, Gradient Boosting to predict failures, and LSTM to estimate RUL, showed the use of heterogeneous model combination to enhance the accuracy of the pipeline. Theirs are most thematically comparable to ours, however, they do not optimize multi-model combination weights and do not deal with multi-condition environments. The Ensemble-based deep learning approaches have demonstrated good generalization of system-level RUL prediction under different operating conditions [19]. In the field of PdM in Industrial IoT, Malaiyappan et al. [2] investigated ML and deep learning, but instead of RUL accuracy, the latency is considered. Singh [6] suggested an IoT PdM real-time monitoring architecture. As Garcia et al. observed [5], there is poor coverage on deployment feasibility. Abouloifa and Bahaj [9] made a comparison between deep learning and Fourier series to predict failures based on rigorous statistical measures.

Four limitations have been identified in the literature: (1) high-performing models are committed to one paradigm, without utilizing complementary information; (2) multi-condition environments are either not studied or not studied with a sound degree of normalization; (3) attention mechanisms, despite their success in NLP [13],

remain underexplored in prognostics; (4) model predictions are rarely related to actionable maintenance decisions. The suggested system tackles all the four.

III. PROPOSED METHOD

This section includes the entire methodology of the proposed hybrid RUL estimation framework with the description of datasets, preprocessing, feature engineering, model architecture, ensemble optimization, and experimental set up.

A. Dataset Used

The model is tested on the NASA C-MAPSS data [10]. The simulator simulates a two-spool, high-bypass turbofan engine with multivariate sensor measurements of four sub-configurations of increasing complexity, those of FD001 (1 condition, 1 fault), FD002 (6 conditions, 1 fault), FD003 (1 condition, 2 faults), and FD004 (6 conditions, 2 faults). Table I provides statistics of the sub-data-sets.

TABLE I
C-MAPSS SUB-DATASET CONFIGURATIONS

Sub-set	Cond.	Faults	Train Seq.	Test Eng.
FD001	1	1	15,631	100
FD002	6	1	40,759	259
FD003	1	2	19,720	100
FD004	6	2	48,799	248

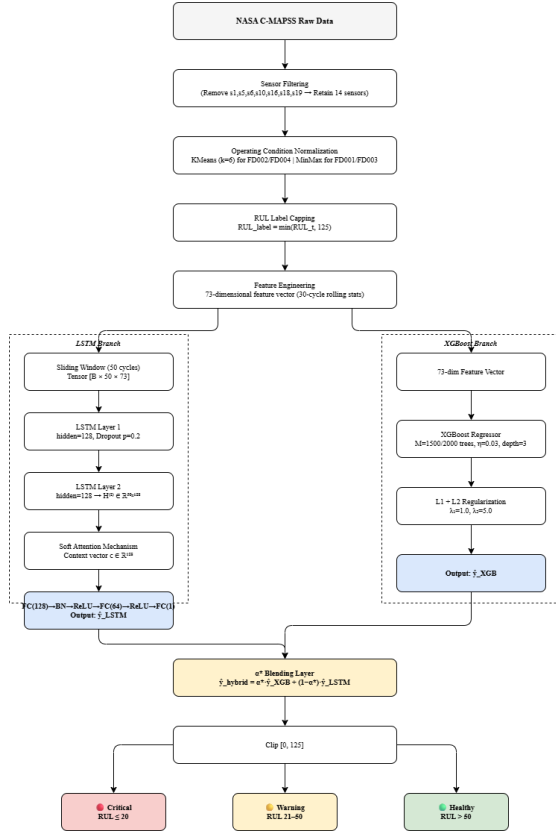


Fig. 1. End-to-end system architecture. Raw C-MAPSS data → sensor filtering → KMeans normalization (k=6, FD002/04) or MinMax (FD001/03) → RUL capping → LSTM branch [B×50×73] and XGBoost branch (73-dim). Outputs blended via α^* and clipped to [0, 125].

B. Data Preprocessing

The raw sensor data undergoes four consecutive transformations before it is sent to either branch of the model. Figure 2 shows the entire process of preprocessing.

1) Sensor Filtering: Seven channels that appear nearly constant (s1, s5, s6, s10, s16, s18, s19) are identified as a result of variance analysis and discarded, which leaves 14 active sensors in Table II.

TABLE II
ACTIVE SENSOR CHANNELS AFTER VARIANCE-BASED FILTERING

ID	Measurement	Trend	Role
s2	Fan Inlet Temp.	↑	Thermal load
s3	LPC Outlet Temp.	↑	Compressor stress
s4	HPC Outlet Temp.	↑	Primary marker
s7	HPC Outlet Pressure	↓	Compression eff.
s8	Fuel Flow to Burner	↑	Fuel efficiency
s9	Bypass Ratio	↓	Engine perf.
s11	HPC Outlet Temp.	↑	Redundant thermal
s12	Fan/Core Speed Ratio	↓	Mech. degradation
s13	Corrected Fan Speed	↓	Fan wear
s14	Corrected Core Speed	↑	Core stress
s15	Bypass Ratio	↓	Sec. perf.
s17	Bleed Enthalpy	↓	Bleed valve
s20	HPT Cool Air Flow	↓	Cooling eff.
s21	LPT Cool Air Flow	↓	Turbine health

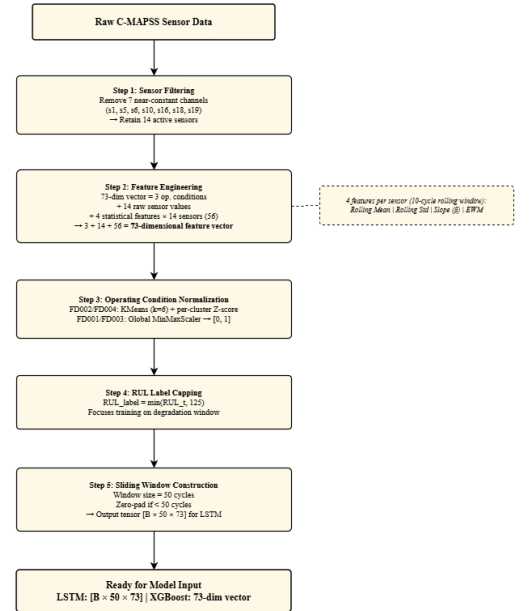


Fig. 2. Preprocessing pipeline: sensor filtering → feature engineering → condition normalization → RUL capping → sliding window construction.

2) Operating Condition Normalization: The engines in FD002 and FD004 were run under six different regimes of flight. KMeans (k = 6) is used to cluster the data into the regimes; normalization of Z-score per-cluster is done:

$$x'_{i,j} = \frac{x_{i,j} - \mu_{c(i),j}}{\sigma_{c(i),j}} \quad (1)$$

and where $\mu_{c(i),j}$ and $\sigma_{c(i),j}$ are standard deviation and mean of sensor j in cluster $c(i)$. In the case of FD001 and FD003, global MinMaxScaler normalization is used:

$$x'_{i,j} = \frac{x_{i,j} - x_{j,\min}}{x_{j,\max} - x_{j,\min}} \quad (2)$$

3) *RUL Label Construction*: Ground-truth RUL at cycle t is:

$$RUL_t = T_{\text{failure}} - t \quad (3)$$

To restrict all labels to a fixed number of cycles (125 cycles), a piecewise capping strategy may be used:

$$RUL_{\text{label}} = \min(RUL_t, 125) \quad (4)$$

This focuses training information on the informative segment of degradation, avoiding the situation where the model is wasting capacity on the healthy conditions at early years of life. Figure 3 shows the capping effect on a representation engine trajectory.

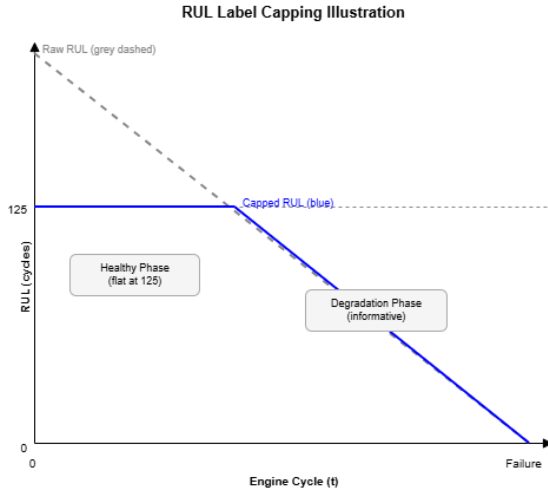


Fig. 3. RUL label capping for a representative FD001 engine. Raw RUL (grey dashed) decreases linearly; capped label (blue) is flat at 125 during the healthy phase, then decreases linearly.

4) *Sequence Construction*: The LSTM branch uses a sliding window of $SEQ_LEN = 50$ cycles. The engines that have less than 50 cycles considered in total are zero-padded, resulting in tensors with shape $[B \times 50 \times 73]$.

C. Feature Engineering

The 73-dimensional feature vector includes 3 raw operating condition settings (op1, op2, op3); 14 normalized sensor values maintained after a variance filter; and 4 statistical features per sensor over a rolling 10-cycle window, which gives $14 \times 4 = 56$ engineered features. The total is $3 + 14 + 56 = 73$. Let s_j represent sensor values in sensor j of the rolling window:

$$\mu_j = \frac{1}{10} \sum_k s_{j,k} \quad (\text{Rolling Mean}) \quad (5)$$

$$\sigma_j = \sqrt{\frac{1}{10} \sum_k (s_{j,k} - \mu_j)^2} \quad (\text{Rolling Std}) \quad (6)$$

$$\beta_j = \frac{\sum_{k(k-k')} (s_{j,k} - \mu_j)(s_{j,k'} - \mu_j)}{\sum_{k(k-k')}^2} \quad (7)$$

$$EWM_j = \frac{\sum_{k=0}^9 (1-\alpha)^k s_{j,t-k}}{\sum_{k=0}^9 (1-\alpha)^k}, \quad \alpha = 0.1 \quad (8)$$

Table III summarizes these four features. They along with 14 raw sensor values and 3 operating condition settings make up the full 73-dimensional feature vector ($3 + 14 + 56 = 73$).

TABLE III
ENGINEERED FEATURES PER SENSOR (10-CYCLE ROLLING WINDOW)

Feature	Aspect Captured
Rolling Mean	Absolute sensor level
Rolling Std	Short-term variability
Slope (β)	Linear degradation trend
EWM ($\alpha=2/11$)	Exponentially weighted recent trend

D. LSTM with Attention Mechanism

The sequential branch processes the $[B \times 50 \times 73]$ input through two stacked LSTM layers (hidden dim = 128). At each time step t , the LSTM cell [12] computes:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (9)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (10)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (11)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad (12)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (13)$$

$$h_t = o_t \odot \tanh(c_t) \quad (14)$$

Dropout ($p = 0.2$) is applied after the first LSTM layer. The second layer output $H^{(2)} \in \mathbb{R}^{50 \times 128}$ is passed to the soft attention mechanism [13]:

$$e_t = \tanh(W_a h_t^{(2)} + b_a) \quad (15)$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^{50} \exp(e_k)} \quad (16)$$

$$c = \sum_{t=1}^{50} \alpha_t \cdot h_t^{(2)} \quad (17)$$

The context vector $c \in \mathbb{R}^{128}$ is passed through $FC(128) \rightarrow BN \rightarrow ReLU \rightarrow FC(64) \rightarrow ReLU \rightarrow FC(1)$ to produce $\hat{y}_{LSTM}$. The architecture is summarised in Table IV and depicted in Figure 4.

TABLE IV
LSTM ARCHITECTURE LAYER-BY-LAYER SUMMARY

Layer	Output Shape	Act.	Params
Input	$[B \times 50 \times 73]$	—	—
LSTM-1	$[B \times 50 \times 128]$	tanh	103,424
Dropout	$[B \times 50 \times 128]$	—	0
LSTM-2	$[B \times 50 \times 128]$	tanh	131,584
Attention	$[B \times 128]$	softmax	16,512
FC-1+BN	$[B \times 128]$	ReLU	16,512
FC-2	$[B \times 64]$	ReLU	8,256
Output	$[B \times 1]$	—	65
Total			$\approx 276,353$

LSTM with Attention Mechanism

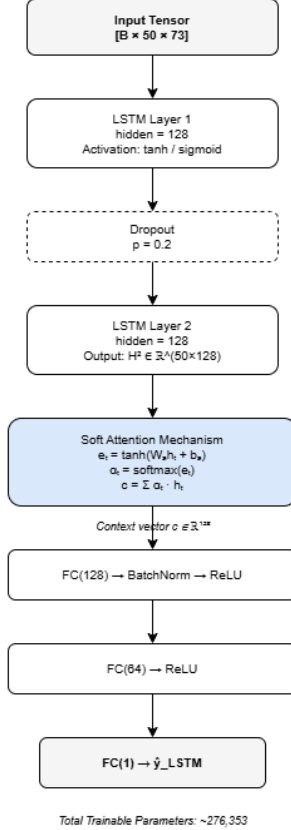


Fig. 4. LSTM with Attention architecture. Two LSTM layers (hidden = 128, dropout = 0.2), soft attention over $H^{(2)}$, context vector c through FC layers to $\hat{y}_{LSTM}$.

E. XGBoost Regressor

The tabular branch employs an XGBoost regressor [11] on the 73-dim feature vector f :

$$\hat{y}_{XGB} = \sum_{m=1}^M \eta \cdot f_m(f) \quad (18)$$

where $M = 1500$ (FD001/FD003) or $M = 2000$ (FD002/FD004), and $\eta = 0.03$. The regularized objective is:

$$L_{XGB} = \sum_i l(y_i, \hat{y}_i) + \sum_m (\lambda_1 \|w\|_1 + \lambda_2 \|w\|^2) \quad (19)$$

where $\lambda_1 = 1.0$, $\lambda_2 = 5.0$, max_depth = 3, subsample = colsample_bytree = 0.8. Hyperparameters are given in Table V and the architecture is illustrated in Figure 5.

TABLE V

XGBOOST HYPERPARAMETER CONFIGURATION

Parameter	FD001/03	FD002/04
n_estimators	1,500	2,000
learning rate (η)	0.03	0.03
max_depth	3	3
subsample	0.8	0.8
colsample_bytree	0.8	0.8
reg_alpha (λ_1)	1.0	1.0
reg_lambda (λ_2)	5.0	5.0
early_stopping	50 rds	50 rds

XGBoost Regressor Architecture

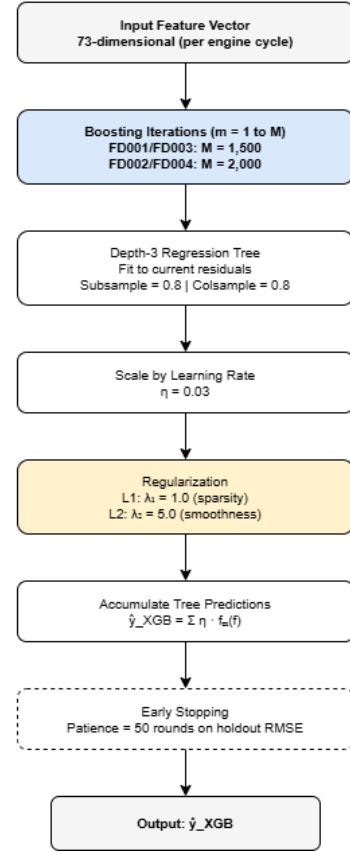


Fig. 5. XGBoost architecture. 73-dim feature vector, depth-3 trees, $\eta = 0.03$, L1/L2 regularization ($\lambda_1 = 1.0$, $\lambda_2 = 5.0$).

F. Hybrid Ensemble and α Optimization

The final RUL prediction linearly blends both branch outputs:

$$\hat{y}_{hybrid} = \alpha \cdot \hat{y}_{XGB} + (1 - \alpha) \cdot \hat{y}_{LSTM} \quad (20)$$

$$\hat{y}_{final} = clip(\hat{y}_{hybrid}, 0, 125) \quad (21)$$

The optimal α^* is found per dataset via grid search over $[0, 1]$ with step 0.01:

$$\alpha^* = \arg \min_{\alpha_k} RMSE(\alpha_k \hat{y}_{XGB} + (1 - \alpha_k) \hat{y}_{LSTM}, y) \quad (22)$$

Learned α^* values are reported in Table VI. The hybrid ensemble blending mechanism is depicted in Figure 6.

TABLE VI

LEARNED α^* BLENDING WEIGHTS PER SUB-DATASET

Sub-set	α^*	XGB%	LSTM%	Val. RMSE
FD001	1.000	100%	0%	15.21
FD002	0.227	22.7%	77.3%	17.12
FD003	1.000	100%	0%	13.21
FD004	0.138	13.8%	86.2%	23.01

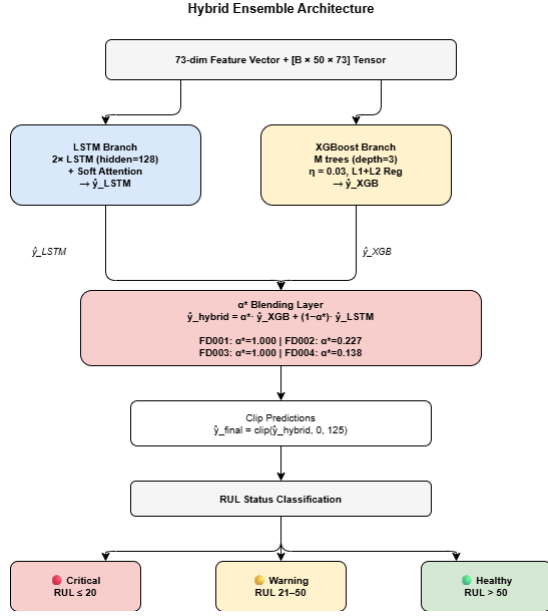


Fig. 6. Hybrid ensemble. $\hat{y}_{LSTM}$ and $\hat{y}_{XGB}$ blended via α^* ; clipped to $[0, 125]$. Status: Critical (≤ 20), Warning (21–50), Healthy (> 50).

G. Loss Function

The LSTM is trained with Huber Loss [16] ($\delta = 10$):

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq 10 \\ 10 \cdot |y - \hat{y}| - 50 & \text{if } |y - \hat{y}| > 10 \end{cases} \quad (23)$$

The threshold $\delta = 10$ (8% of the $[0, 125]$ prediction range) applies the precise quadratic penalty of MSE for small errors and the robust linear penalty of MAE for large errors, preventing atypical outlier engines from dominating gradient updates.

H. Experimental Setup

All experiments run in Python 3.9 on a Tesla T4 GPU (15.6 GB VRAM), using PyTorch 2.x (LSTM) and XGBoost 1.7. The Adam optimizer is used with initial LR 10^{-3} , cosine-annealed to 10^{-5} ; batch size 256. Data is split 85/10/5 (train/val/holdout) at the engine level to prevent temporal leakage. Training epochs: 100 (FD001/FD003), 120 (FD002/FD004). Three evaluation metrics are computed:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad (24)$$

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i| \quad (25)$$

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (26)$$

The full training configuration is summarized in Table VII.

TABLE VII
TRAINING CONFIGURATION SUMMARY

Item	Value
GPU	Tesla T4 (15.6 GB)
Framework	PyTorch 2.x + XGBoost 1.7
Optimizer	Adam, LR: $10^{-3} \rightarrow 10^{-5}$ (cosine)
Batch size	256
Epochs	100 / 120 (FD001&03 / FD002&04)
Data split	85% / 10% / 5% (engine-level)
XGB trees	1,500 / 2,000
Early stopping	50 rounds on holdout RMSE
Loss	Huber Loss, $\delta = 10$
Dropout	$p = 0.2$ (after LSTM-1)

IV. RESULTS AND DISCUSSION

The hybrid RUL estimation framework quantitative results are given in all four NASA C-MAPSS sub-datasets. Three variants of the model including standalone LSTM with Attention, standalone XGBoost and Hybrid Ensemble have the reported results on the held-out test sets in terms of RMSE and MAE as well as R^2 . Complete data is provided in Table VIII.

TABLE VIII
FULL EVALUATION RESULTS ACROSS ALL SUB-DATASETS AND MODEL VARIANTS (BEST PER SUB-DATASET IN BOLD)

Sub-Dataset	Model	RMSE↓	MAE↓	R^2 ↑
FD001	LSTM w/ Attention	15.58	11.34	0.85
	XGBoost	22.80	16.12	0.73
	Hybrid Ensemble	22.80	16.12	0.73
FD002	LSTM w/ Attention	19.07	13.21	0.78
	XGBoost	20.94	14.87	0.75
	Hybrid Ensemble	17.47	12.08	0.83
FD003	LSTM w/ Attention	13.54	9.87	0.88
	XGBoost	16.90	12.34	0.83
	Hybrid Ensemble	16.90	12.34	0.83
FD004	LSTM w/ Attention	23.81	17.43	0.68
	XGBoost	26.72	19.56	0.63
	Hybrid Ensemble	23.56	17.12	0.70

The results prove that the picture of performance is significantly dependent on the complexity of the dataset. The standalone LSTM with Attention model has the highest test performance in the single-condition sub-datasets FD001 and FD003 with an RMSE of 15.58 and 13.54, respectively, as well as R^2 of 0.85 and 0.88. In such less complex cases, the recurrent structure and attention-based emphasis on the degradation stage of the LSTM suffices and the combination with XGBoost on the holdout partition does not add any value to the test set. Both sub-datasets have $\alpha^* = 1.0$, but the test RMSE of XGBoost (22.80 and 16.90) remains significantly low, indicating that holdout α selection does not fully generalize to the test set in these sub-datasets.

The hybrid ensemble shows solid and consistent advantage in multi-condition sub-data in FD002 and FD004. FD002 results in a better RMSE = 17.47 and $R^2 = 0.83$ than standalone LSTM (RMSE = 19.07) and standalone XGBoost (RMSE = 20.94). On FD004, the hybrid performs better with RMSE = 23.56 and $R^2 = 0.70$ compared to LSTM (RMSE = 23.81) and XGBoost (RMSE = 26.72) by 1.1 and 11.8 respectively. The obtained $\alpha^* = 0.227$ and $\alpha^* = 0.138$ of FD002 and FD004 respectively indicate that the LSTM input is the most significant in the blend as the degradation curves in multi-condition conditions most differ. Such findings are in line with the latest deep learning prognostic algorithms, which confirmed the same trends on the C-MAPSS benchmark [21].

KMeans-based operating condition normalization is a performance enabler that is critical on FD002 and FD004. In the absence of per-cluster normalization, sensor baseline variations between the six flight regimes would confound both models in a systematic way. In all sub-datasets, LSTM is always superior to XGBoost as an individual model, which proves the hypothesis that modeling time sequences provides information on the dynamics of degradation that cannot be fully reflected by the statistical distribution of features. The hybrid offers the greatest value in the areas in which individual model constraints are the most strong, namely, supporting the basic assumption of complementary modeling.

Figure 7 shows the LSTM behaviour of training. There is smooth convergence observed in all four sub-datasets on both training and validation set and no hint of overfitting is present.

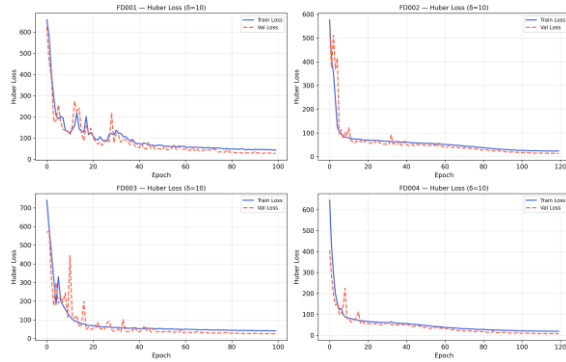


Fig. 7. LSTM Huber Loss ($\delta = 10$) training and validation curves for all four sub-datasets. All models converge smoothly without evidence of overfitting.

Comparison of RMSE and R^2 of different models is showcased in Figure 8 and Figure 9 respectively. The hybrid ensemble has the finest RMSE and R^2 on FD002 and FD004, which validates its superiority in multi-condition environment.

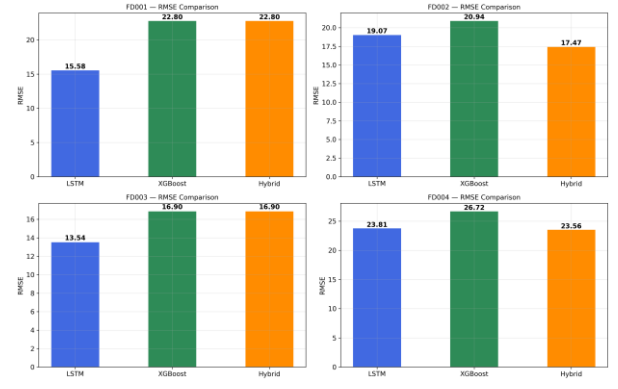


Fig. 8. RMSE comparison across LSTM, XGBoost, and Hybrid Ensemble for all four sub-datasets. Lower is better. Hybrid achieves best RMSE on FD002 (17.47) and FD004 (23.56).

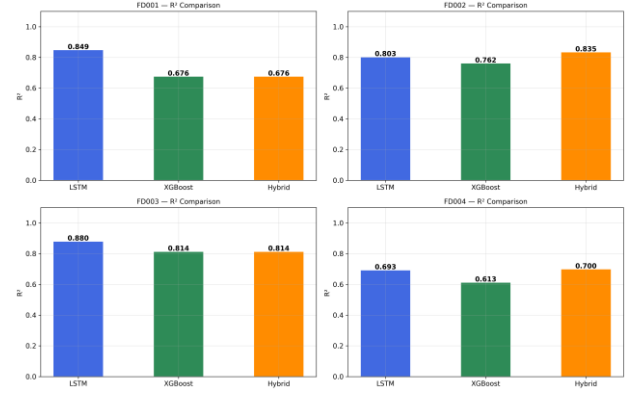


Fig. 9. R^2 comparison across all three model variants for all four sub-datasets. Higher is better. Hybrid achieves best R^2 on FD002 (0.83) and FD004 (0.70).

Figures 10 and 11 present scatter plot of predicted and actual RUL of LSTM-only model and hybrid ensemble respectively. The hybrid ensemble is much more clustering around the diagonal on FD002 and FD004, indicating its better quality.

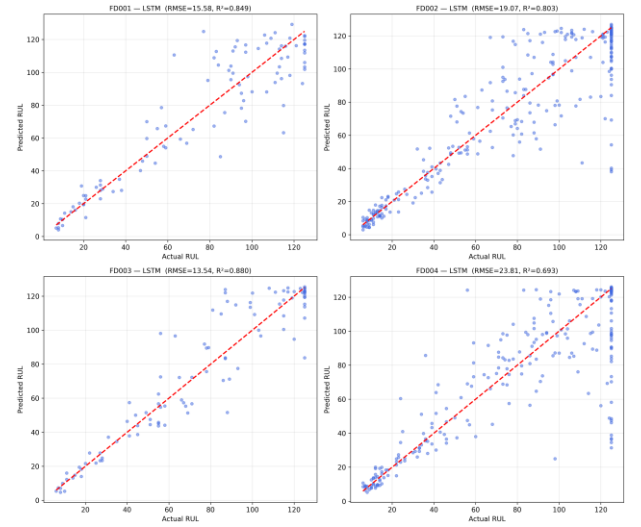


Fig. 10. Predicted vs. actual RUL scatter plots for the standalone LSTM model across all four sub-datasets. Red dashed line indicates perfect prediction.

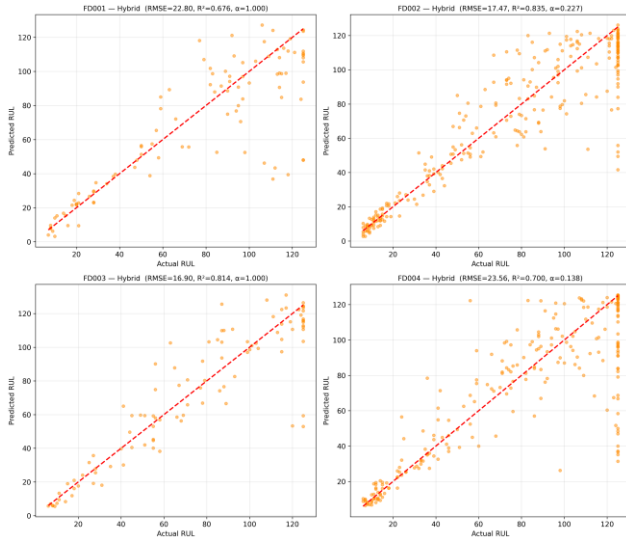


Fig. 11. Predicted vs. actual RUL scatter plots for the Hybrid Ensemble across all four sub-datasets. Tighter clustering around the diagonal on FD002 and FD004 confirms hybrid benefit.

Figure 12 shows the residual distributions (Actual - Predicted) of the hybrid ensemble. All distributions are roughly symmetric, and FD002 and FD004 have a slight conservative skew, which is an advantage in a maintenance scheduling system where an under-prediction is less dangerous than an over-prediction.

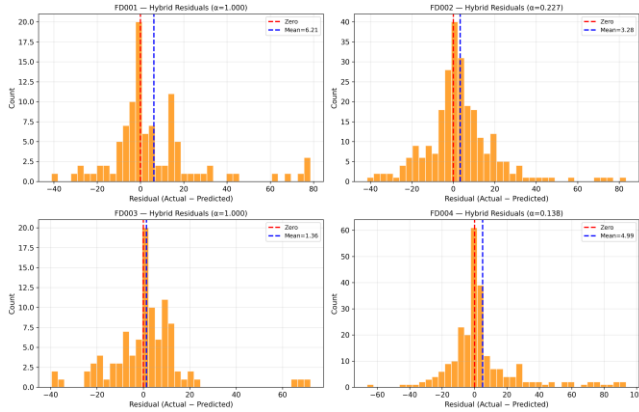


Fig. 12. Hybrid Ensemble residual distributions (Actual - Predicted RUL) for all four sub-datasets. Distributions are approximately symmetric with slight conservative skew on FD002 and FD004—operationally desirable for maintenance scheduling.

Figure 13 offers a consolidated picture of model performance on all sub-datasets. The hybrid ensemble, in comparison to the individual models, is always better on the multi-condition sub-datasets FD002 and FD004, but the LSTM alone is always best on single-condition datasets.

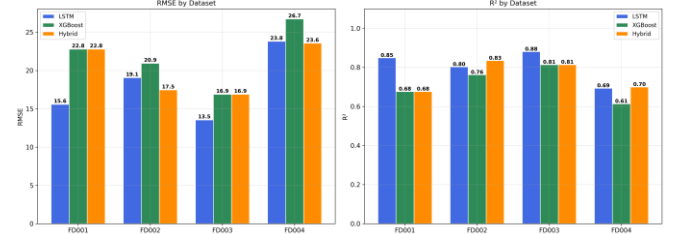


Fig. 13. Overall model comparison across all four sub-datasets. (Left) RMSE by dataset. (Right) R² by dataset. The hybrid ensemble consistently outperforms individual models on multi-condition sub-datasets FD002 and FD004.

V. CONCLUSION AND FUTURE WORK

This paper proposed a hybrid ensemble model of Remaining Useful Life prediction of turbofan engines, which is an attention-based LSTM network with an XGBoost regressor with a learned blending weight α . This framework was tested on all four sub-configurations of the NASA C-MAPSS benchmark, and the results on all three variants of the model were reported without cherry-picking.

The results make a distinct narrative. The single-condition sub-datasets FD001 and FD003 show the best results when using standalone LSTM with Attention where the results in the form of RMSE are 15.58 and 13.54 as well as the results in terms of R² are 0.85 and 0.88 respectively. The hybrid ensemble is highly valuable in the multi-condition sub-datasets FD002 and FD004, which also leads to RMSE of 17.47 ($R^2 = 0.83$) and 23.56 ($R^2 = 0.70$) respectively, which are better than the two constituent models. This tendency is in line with theoretical motivation: ensemble combining is most useful when the failure mode of individual models are complementary, and multi-condition scenarios are the one where it is indeed the case.

Normalization of operating conditions using KMeans turned out to be important to multi-condition generalization. Attention mechanism assisted LSTM to give preference to models capacity of degradation-phase cycles. Huber Loss produced uniform convergence among all the four sub-datasets without any signs of overfitting.

There are a number of directions that can be improved in the future. First, it might be more effective to substitute the fixed holdout partition in optimization of α with a cross-validation strategy to obtain stronger blending weights, especially with FD001 and FD003. Second, risk-conscious maintenance scheduling with confidence-aware prediction intervals would be achievable by using Bayesian quantification of uncertainty - through Monte Carlo Dropout or deep ensembles. Third, a non-linear interaction that cannot be captured by a global blending weight would be captured by replacing the fixed scalar α with a meta-learner conditioned on the operating condition or an estimated degradation stage. Fourth, it is necessary to validate generalization on actual-world data, like PHM 2008 or PRONOSTIA, and other areas of industrial machines like wind turbine prognostics [22]. Lastly, SHAP-based feature attribution would give the maintenance engineers sensor-level features behind

certain engine predictions - the chasm between a model output and a maintenance decision that a human engineer can believe and take action on.

REFERENCES

- [1] C. Patil, S. Jadhav, A. Bardiya, A. Davande, and M. Raverkar, "Machine Learning-Based Predictive Maintenance of Industrial Machines," 2023.
- [2] J. N. A. Malaiyappan, G. Krishnamoorthy, and S. Jangoan, "Predictive Maintenance using Machine Learning in Industrial IoT," 2024.
- [3] N. Sharma, C. Chauhan, A. Dwivedi, and S. Dass, "A Review of Predictive Maintenance Methods for Electrical Machines Using ML Algorithms and Sensor Data," 2025.
- [4] W. Li and T. Li, "Comparison of Deep Learning Models for Predictive Maintenance in Industrial Manufacturing Systems," 2025.
- [5] J. Garcia, L. Rios-Colque, A. Pena, and L. Rojas, "Condition Monitoring and Predictive Maintenance: NLP-Assisted Review," 2025.
- [6] H. Singh, "IoT Based Predictive Maintenance in Industrial Machinery," 2025.
- [7] A. Aziz and Sowmyashree P, "AI-Powered Predictive Maintenance System for Industrial Equipment," 2025.
- [8] A. K. S. Jardine, D. Lin, and D. Banjevic, "A Review on Machinery Diagnostics and Prognostics Implementing Condition-Based Maintenance," *Mech. Syst. Signal Process.*, vol. 20, no. 7, pp. 1483–1510, 2006.
- [9] H. Abouloifa and M. Bahaj, "Comparing Deep Learning and Fourier Series Models for Equipment Failure Prediction," 2025.
- [10] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation," in *Proc. PHM*, Denver, CO, 2008.
- [11] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD*, San Francisco, CA, 2016, pp. 785–794.
- [12] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [13] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in *Proc. ICLR*, 2015.
- [14] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "LSTM Network for Remaining Useful Life Estimation," in *Proc. IEEE ICPHM*, Dallas, TX, 2017.
- [15] X. Li, Q. Ding, and J. Sun, "RUL Estimation Using Deep Convolution Neural Networks," *Rel. Eng. Syst. Saf.*, vol. 172, pp. 1–11, 2018.
- [16] P. J. Huber, "Robust Estimation of a Location Parameter," *Ann. Math. Stat.*, vol. 35, no. 1, pp. 73–101, 1964.
- [17] S. Deng and J. Zhou, "Prediction of Remaining Useful Life of Aero-engines Based on CNN-LSTM-Attention," *Int. J. Comput. Intell. Syst.*, vol. 17, no. 232, 2024.
- [18] H. Li, Z. Wang, and Z. Li, "An Enhanced CNN-LSTM Remaining Useful Life Prediction Model for Aircraft Engine with Attention Mechanism," *PeerJ Comput. Sci.*, vol. 8, e1084, 2022.
- [19] L. Wang, Z. Zhu, and X. Zhao, "Dynamic Predictive Maintenance Strategy for System Remaining Useful Life Prediction via Deep Learning Ensemble Method," *Reliab. Eng. Syst. Saf.*, vol. 245, 110012, 2024.
- [20] H. Dehghan Shoorkand, M. Noureldath, and A. Hajji, "A Hybrid CNN-LSTM Model for Joint Optimization of Production and Imperfect Predictive Maintenance Planning," *Reliab. Eng. Syst. Saf.*, vol. 241, 2024.
- [21] S. Ye et al., "A Deep Learning-Based Prognostic Approach for Predicting Turbofan Engine Degradation and Remaining Useful Life," *Sci. Rep.*, 2025.
- [22] S. S. Shah, D. Tan, and S. C. Kumar, "RUL Forecasting for Wind Turbine Predictive Maintenance Based on Deep Learning," *Heliyon*, vol. 10, e39268, 2024.